

Complete AWS Deployment — Empty Account to Running App

Standalone. Nothing assumed, nothing referenced elsewhere. Start with an empty AWS account, finish with the application live on HTTPS.

Every command in order. Every value explained — no `<PLACEHOLDER>` left dangling.

Total time: ~90 minutes, most of it waiting on AWS. **Cost:** ~\$5-8/day while running. Phase 12 tears it all down.

Phase map

Phase	What	Time
0	Prerequisites + AWS account	15 min
1	Fix the code (7 required changes)	20 min
2	Cluster + networking	20 min
3	Cluster add-ons	10 min
4	ECR + build + push images	15 min
5	TLS certificate + DNS	10 min
6	Secrets	5 min
7	Deploy the application	10 min
8	Expose via ALB	5 min
9	Verify	5 min
10	Day-2 operations	—
11	Cost control	—
12	Teardown	15 min

Set these once and keep the terminal open — every phase uses them:

```
export AWS_REGION=ap-south-1
export CLUSTER=ecommerce-cluster
export DOMAIN=yourdomain.com # a domain you actually own
export PROJECT=$HOME/Desktop/ecommerce
cd $PROJECT
```

PHASE 0 — Prerequisites

0.1 Install the tools

```
# Java 17
sudo apt install -y openjdk-17-jdk maven git jq unzip

# AWS CLI v2
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q awscliv2.zip && sudo ./aws/install && rm -rf aws awscliv2.zip

# kubectl
curl -LO "https://dl.k8s.io/release/$(curl -Ls https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -m 0755 kubectl /usr/local/bin/kubectl && rm kubectl

# eksctl
curl -sL "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_linux_amd64.tar.gz" \
| tar xz -C /tmp && sudo mv /tmp/eksctl /usr/local/bin

# helm
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
```

```
# docker
sudo apt install -y docker.io
sudo usermod -aG docker $USER      # log out and back in after this
```

Verify all of them:

```
java -version && mvn -v && aws --version && kubectl version --client \
&& eksctl version && helm version --short && docker --version
```

0.2 AWS credentials

In the AWS Console: **IAM** → **Users** → **Create user** → attach `AdministratorAccess` → **Security credentials** → **Create access key** → **CLI**.

```
aws configure
# AWS Access Key ID: AKIA...
# AWS Secret Access Key: ...
# Default region name: ap-south-1
# Default output format: json
```

Confirm it works — this must succeed before continuing:

```
aws sts get-caller-identity

{ "UserId": "AIDA...", "Account": "123456789012", "Arn": "arn:aws:iam::123456789012:user/you" }
```

Save your account ID — several later commands need it:

```
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export REGISTRY=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com
echo $REGISTRY
```

0.3 A domain you own

Required for HTTPS. `.local` is **not** a real TLD — ACM can never issue a certificate for it, and the HTTPS listener will fail.

- Already own one elsewhere? Fine — you'll point its nameservers at Route 53 in Phase 5.
- Don't own one? **Route 53** → **Registered domains** → **Register** (~\$12/year, `.click` and `.link` are cheapest).

PHASE 1 — Fix the code

The repository will not deploy to AWS as-is. Seven changes, all required. Do them now — discovering them mid-deploy wastes an hour.

1.1 Registry URL

`scripts/03-ecr.sh` line 20 hardcodes a local registry:

```
sed -i 's|^REGISTRY=localhost:5100|REGISTRY=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com|' scripts/03-ecr.sh
```

Also add the account lookup just above it:

```
sed -i '/^REGISTRY=/i ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)' scripts/03-ecr.sh
```

Why: `localhost:5100` inside a pod means *that node*, which runs no registry. Every service would sit in `ImagePullBackOff`.

1.2 StorageClass on every volume claim

EKS ships **no default StorageClass**. The 7 `volumeClaimTemplates` (5 Postgres, Redis, Kafka) would stay `Pending` forever and the deploy would abort.

```
sed -i '/accessModes:/i\    storageClassName: gp3' \  
k8s/postgres/postgres.yaml k8s/redis/redis.yaml k8s/kafka/kafka.yaml
```

```
grep -A2 volumeClaimTemplates k8s/postgres/postgres.yaml | head -8
```

You want each to read:

```
volumeClaimTemplates:  
- metadata:  
  name: data  
  spec:  
    storageClassName: gp3  
    accessModes: [ReadWriteOnce]
```

1.3 Remove the local AWS endpoint override

`k8s/services/configmap.yaml` points the AWS SDK at a local emulator that won't exist in the cluster. `notification-service` uses it for SES.

```
sed -i '/AWS_ENDPOINT_URL/d' k8s/services/configmap.yaml
grep -A8 "^data:" k8s/services/configmap.yaml
```

Should now be:

```
data:
  KAFKA_BOOTSTRAP_SERVERS: "kafka-0.kafka.ecommerce.svc.cluster.local:9092"
  REDIS_HOST: "redis-0.redis.ecommerce.svc.cluster.local"
  REDIS_PORT: "6379"
  AWS_DEFAULT_REGION: "ap-south-1"
  SPRING_PROFILE: "prod"
```

1.4 Kong Service → ClusterIP

k8s/kong/kong.yaml has `type: LoadBalancer` plus ALB annotations. Left as-is you get a **second, duplicate ALB** alongside the Ingress one.

```
sed -i '/service.beta.kubernetes.io\/aws-load-balancer-type/d;  
      /service.beta.kubernetes.io\/aws-load-balancer-scheme/d' k8s/kong/kong.yaml  
sed -i 's/^ type: LoadBalancer/ type: ClusterIP/' k8s/kong/kong.yaml
```


1.5 Don't let the deploy script edit tracked files

`scripts/05-deploy.sh` line 31 uses `sed -i` on `configmap.yaml`, permanently destroying the placeholder and dirtying your working tree.

```
sed -i 's|^sed -i "s\|REGISTRY\|\\$REGISTRY\|g" k8s/services/configmap.yaml|sed "s\|REGISTRY\|\\$REGISTRY\|g" k8s/services/configmap.yaml \| ku
```

If that substitution looks fragile, just edit the file by hand — change line 31 from:

```
sed -i "s|REGISTRY|$REGISTRY|g" k8s/services/configmap.yaml
kubectl apply -f k8s/services/configmap.yaml
```

to:

```
sed "s|REGISTRY|$REGISTRY|g" k8s/services/configmap.yaml | kubectl apply -f -
```

1.6 Remove the manual target registration

`scripts/05-deploy.sh` lines ~70-80 snapshot Kong's pod IPs into an ALB target group **once**. After the first pod restart those IPs are dead and you get 503s. The ALB controller (Phase 3) does this continuously instead.

Open `scripts/05-deploy.sh` and delete the whole block from `echo "=== [6/8] Register Kong in ALB Target Group ==="` through the `describe-target-health` call.

1.7 CPU requests so HPA works

The manifests define HPAs. HPA computes a **percentage of requested CPU** — with no `resources.requests.cpu` there's no denominator, so it reads `<unknown>` and never scales.

Check whether they're already set:

```
grep -c "cpu:" k8s/services/deployments.yaml
```

If that returns 0, add to each container spec:

```
resources:
  requests:
    cpu: 200m
    memory: 512Mi
  limits:
    cpu: 1000m
    memory: 1Gi
```

1.8 Commit

```
git checkout -b aws-deploy
git add -A && git commit -m "chore: prepare for AWS deployment"
```

PHASE 2 — Cluster and networking

`eksctl` builds the VPC, subnets, IGW, NAT, route tables, IAM roles, node groups and — importantly — **applies the `kubernetes.io/role/*` subnet tags automatically**. Doing this by hand with `aws ec2` is ~25 commands and the tags are the most commonly missed step.

2.1 Cluster config

```
cat > cluster.yaml <<EOF
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: $CLUSTER
  region: $AWS_REGION
  version: "1.29"

vpc:
  cidr: 10.0.0.0/16
  nat:
    gateway: HighlyAvailable # one NAT per AZ – survives an AZ failure
  clusterEndpoints:
    publicAccess: true
    privateAccess: true

iam:
  withOIDC: true # required for IRSA (add-ons in Phase 3)

managedNodeGroups:
  - name: general
    instanceType: t3.large
    minSize: 2
    maxSize: 6
    desiredCapacity: 3
    volumeSize: 30
    privateNetworking: true # nodes get no public IP
    labels: { role: general }
    iam:
      withAddonPolicies:
        ebs: true
        cloudWatch: true
        autoScaler: true

  - name: high-mem
    instanceType: r5.large
    minSize: 1
    maxSize: 4
    desiredCapacity: 2
    volumeSize: 40
    privateNetworking: true
    labels: { role: high-memory }
    iam:
      withAddonPolicies:
        ebs: true
        cloudWatch: true

cloudWatch:
  clusterLogging:
    enableTypes: ["api", "audit", "authenticator", "controllerManager", "scheduler"]
EOF
```

`nat.gateway: HighlyAvailable` creates one NAT per AZ. `Single` is ~\$32/month cheaper but means an AZ-a failure takes internet away from AZ-b as well — which defeats the point of spreading nodes across AZs.

2.2 Create it

```
eksctl create cluster -f cluster.yaml
```

🕒 **15-20 minutes.** It prints progress; leave it alone.

2.3 Verify

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-x-x.ap-south-1.compute.internal	Ready	<none>	2m	v1.29.x
... (5 nodes: 3 general + 2 high-mem)				

```
kubectl get nodes -L role # confirm the labels applied
```

```
aws eks describe-cluster --name $CLUSTER --query 'cluster.status' # "ACTIVE"
```

Confirm the subnet tags exist (this is what the ALB controller needs):

```
aws ec2 describe-subnets \
  --filters "Name=tag:kubernetes.io/cluster/$CLUSTER,Values=shared" \
  --query 'Subnets[].[ID:SubnetId,Tags:Tags[?starts_with(Key,`kubernetes.io/role`)].Key]' \
  --output table
```

You should see `kubernetes.io/role/elb` on public subnets and `kubernetes.io/role/internal-elb` on private ones.

PHASE 3 — Cluster add-ons

Three add-ons. Without them: no storage, no autoscaling, no load balancer.

3.1 EBS CSI driver — storage

```
eksctl create iamserviceaccount \
  --name ebs-csi-controller-sa --namespace kube-system \
  --cluster $CLUSTER --region $AWS_REGION \
  --attach-policy-arn arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy \
  --approve --role-only --role-name AmazonEKS_EBS_CSI_DriverRole

eksctl create addon --name aws-ebs-csi-driver \
  --cluster $CLUSTER --region $AWS_REGION \
  --service-account-role-arn arn:aws:iam::$ACCOUNT_ID:role/AmazonEKS_EBS_CSI_DriverRole \
  --force
```

The `gp3` `StorageClass` (referenced in Phase 1.2):

```
kubectl apply -f - <<'EOF'
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: gp3
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: ebs.csi.aws.com
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
parameters:
  type: gp3
  encrypted: "true"
EOF
```

```
kubectl get storageclass
```

`WaitForFirstConsumer` **matters**. It delays creating the EBS volume until the pod is scheduled, so the volume lands in the **same AZ** as the pod. Without it you get volumes in AZ-a and pods in AZ-b that can never attach.

3.2 metrics-server — for HPA

```
kubect1 apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubect1 -n kube-system rollout status deploy/metrics-server --timeout=120s
kubect1 top nodes          # must return numbers, not an error
```

EKS does not ship this. Without it every HPA reads `<unknown>/70%` and silently never scales — no error, no event.

3.3 AWS Load Balancer Controller

This is what turns an Ingress object into a real ALB, and **keeps the target list in sync** as pods restart and scale.

```
# permissions
curl -sO https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.7.1/docs/install/iam_policy.json
aws iam create-policy --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file:///iam_policy.json 2>/dev/null || echo "policy already exists"

# identity — links a K8s ServiceAccount to an IAM role (IRSA)
eksctl create iamserviceaccount \
  --cluster $CLUSTER --region $AWS_REGION \
  --namespace kube-system --name aws-load-balancer-controller \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --attach-policy-arn arn:aws:iam::$ACCOUNT_ID:policy/AWSLoadBalancerControllerIAMPolicy \
  --approve

# the controller
helm repo add eks https://aws.github.io/eks-charts && helm repo update
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=$CLUSTER \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=$AWS_REGION

kubectl -n kube-system rollout status deploy/aws-load-balancer-controller --timeout=120s
```

Verify:

```
kubectl -n kube-system logs deploy/aws-load-balancer-controller --tail=20
```

No AccessDenied, no failed to get VPC ID.

IRSA in one line: the pod gets a signed token from Kubernetes, trades it with AWS STS for temporary credentials, and they auto-rotate hourly. No AWS keys stored anywhere.

PHASE 4 — Build and push images

4.1 Create the repositories

```
for SVC in user-service product-service order-service \
  inventory-service payment-service notification-service; do
  aws ecr create-repository --repository-name ecommerce/$SVC \
    --region $AWS_REGION \
    --image-scanning-configuration scanOnPush=true \
    --encryption-configuration encryptionType=AES256 2>/dev/null \
    && echo "created ecommerce/$SVC" || echo "exists ecommerce/$SVC"
done
```

Lifecycle policy — keep only the last 10 images per repo (otherwise storage grows forever):

```
for SVC in user-service product-service order-service \
  inventory-service payment-service notification-service; do
  aws ecr put-lifecycle-policy --repository-name ecommerce/$SVC --region $AWS_REGION \
    --lifecycle-policy-text '{"rules":[{"rulePriority":1,"description":"Keep 10","selection":{"tagStatus":"any","countType":"imageCountMoreTf
done
```

4.2 Build and push

```
export TAG=$(git rev-parse --short HEAD)
echo "Deploying tag: $TAG"

aws ecr get-login-password --region $AWS_REGION \
| docker login --username AWS --password-stdin $REGISTRY

mvn -B -T 1C package -DskipTests

for SVC in user-service product-service order-service \
    inventory-service payment-service notification-service; do
    docker build -t $REGISTRY/ecommerce/$SVC:$TAG $SVC/
    docker push $REGISTRY/ecommerce/$SVC:$TAG
    echo "✅ $SVC:$TAG"
done
```

🕒 ~10-15 minutes on a first run.

Verify:

```
aws ecr describe-images --repository-name ecommerce/order-service \
--region $AWS_REGION --query 'imageDetails[].imageTags' --output text
```

Tag by git SHA, never latest. `latest` means something different tomorrow, so there is nothing to roll back to. The SHA ties a running container to an exact commit.

PHASE 5 — TLS certificate and DNS

5.1 Route 53 hosted zone

```
aws route53 create-hosted-zone --name $DOMAIN \
--caller-reference $(date +%s) \
--query 'HostedZone.Id' --output text

export ZONE_ID=$(aws route53 list-hosted-zones-by-name --dns-name $DOMAIN \
--query 'HostedZones[0].Id' --output text | cut -d/ -f3)
echo $ZONE_ID
```

Point your domain at these nameservers (at your registrar, if the domain wasn't registered in Route 53):

```
aws route53 get-hosted-zone --id $ZONE_ID --query 'DelegationSet.NameServers' --output table
```

🕒 DNS delegation can take up to 48 hours, though it's usually under an hour.

5.2 Request the certificate

```
export CERT_ARN=$(aws acm request-certificate \  
  --domain-name $DOMAIN \  
  --subject-alternative-names ".*$DOMAIN" \  
  --validation-method DNS \  
  --region $AWS_REGION \  
  --query CertificateArn --output text)  
echo $CERT_ARN
```

5.3 Validate it

ACM needs a DNS record proving you control the domain:

```
aws acm describe-certificate --certificate-arn $CERT_ARN --region $AWS_REGION \
--query 'Certificate.DomainValidationOptions[0].ResourceRecord'

{ "Name": "_a1b2c3.yourdomain.com.", "Type": "CNAME", "Value": "_x9y8z7.acm-validations.aws." }
```

Create it automatically:

```
REC=$(aws acm describe-certificate --certificate-arn $CERT_ARN --region $AWS_REGION \
--query 'Certificate.DomainValidationOptions[0].ResourceRecord' --output json)

aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-batch "{
  \"Changes\": [{\"Action\": \"UPSERT\", \"ResourceRecordSet\": {
    \"Name\": $(echo $REC | jq .Name),
    \"Type\": \"CNAME\", \"TTL\": 300,
    \"ResourceRecords\": [{\"Value\": $(echo $REC | jq .Value)}}]}]"
```

Wait for issuance:

```
aws acm wait certificate-validated --certificate-arn $CERT_ARN --region $AWS_REGION
aws acm describe-certificate --certificate-arn $CERT_ARN --region $AWS_REGION \
--query 'Certificate.Status' --output text      # ISSUED
```

🕒 ~5 minutes once the DNS record propagates.

PHASE 6 — Secrets

6.1 Fill them in

```
cp k8s/services/secrets.example.yaml k8s/services/secrets.yaml
```

Generate real values:

```
export JWT=$(openssl rand -base64 48)      # must be ≥32 chars
export REDIS_PW=$(openssl rand -base64 24)
export DB_PW=$(openssl rand -base64 24)

sed -i "s|JWT_SECRET: \".*\"|JWT_SECRET: \"$JWT\"|" k8s/services/secrets.yaml
sed -i "s|REDIS_PASSWORD: \".*\"|REDIS_PASSWORD: \"$REDIS_PW\"|" k8s/services/secrets.yaml
sed -i "s|_DB_USER: \".*\"|_DB_USER: \"postgres\"|g" k8s/services/secrets.yaml
sed -i "s|_DB_PASS: \".*\"|_DB_PASS: \"$DB_PW\"|g" k8s/services/secrets.yaml
```

no AWS keys in Secrets – IRSA handles it (6.3)

```
sed -i '/AWS_ACCESS_KEY_ID/d;/AWS_SECRET_ACCESS_KEY/d' k8s/services/secrets.yaml
```

```
grep -c CHANGE_ME k8s/services/secrets.yaml      # must print 0
```

The Redis password must match `redis.yaml` `'s --requirepass :`

```
grep -n "requirepass" k8s/redis/redis.yaml
```

If it's hardcoded there rather than read from the Secret, update it to match.

`secrets.yaml` is gitignored. Never commit it — only `secrets.example.yaml` belongs in the repo.

6.2 Verify SES for notification emails

```
aws ses verify-email-identity --email-address noreply@$DOMAIN --region $AWS_REGION
```

Check your inbox and click the link. **SES starts in sandbox mode** — it will only send to *verified* addresses until you request production access (**SES** → **Account dashboard** → **Request production access**).

6.3 Give notification-service SES access via IRSA

```
eksctl create iamserviceaccount \
  --name notification-sa --namespace ecommerce \
  --cluster $CLUSTER --region $AWS_REGION \
  --attach-policy-arn arn:aws:iam::aws:policy/AmazonSESEFullAccess \
  --approve
```

This fails if the `ecommerce` namespace doesn't exist yet. Either create it first (`kubect1 create ns ecommerce`) or run this right after Phase 7.1.

Then add to the notification-service Deployment in `k8s/services/deployments.yaml` :

```
spec:
  template:
    spec:
      serviceAccountName: notification-sa
```

PHASE 7 — Deploy the application

Order matters and each step waits for the previous one.

7.1 Namespaces, secrets, config

```
kubectl apply -f k8s/namespace/namespace.yaml
kubectl apply -f k8s/services/secrets.yaml
sed "s|REGISTRY|$REGISTRY|g" k8s/services/configmap.yaml | kubectl apply -f -

kubectl get ns ecommerce monitoring
kubectl -n ecommerce get secret,configmap
```

Why first: pods read Secrets and ConfigMaps **at startup**. Deploy them after and every pod crash-loops.

7.2 Data layer

```
kubectl apply -f k8s/postgres/postgres.yaml
kubectl apply -f k8s/redis/redis.yaml
kubectl apply -f k8s/kafka/kafka.yaml
```

Watch the volumes bind — this is where a missing StorageClass shows up:

```
kubectl -n ecommerce get pvc -w      # all must reach Bound

for a in postgres-user postgres-product postgres-order postgres-inventory postgres-payment redis kafka; do
  kubectl -n ecommerce wait --for=condition=ready pod -l app=$a --timeout=300s && echo "✅ $a"
done
```

🕒 Kafka takes ~60-90s.

Confirm the topics exist — the manifests set `KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"`, so a Job creates them explicitly:

```
kubectl -n ecommerce get job kafka-topics-init
kubectl -n ecommerce exec kafka-0 -- kafka-topics --bootstrap-server localhost:9092 --list

inventory.updated
order.cancelled
order.created
payment.processed
product.created
```

Without these five topics the saga cannot run — orders would stay `PENDING` forever.

7.3 Kong gateway

```
kubectl apply -f k8s/kong/kong.yaml
kubectl -n ecommerce wait --for=condition=ready pod -l app=kong --timeout=180s
kubectl -n ecommerce get svc kong-proxy # must be ClusterIP, not LoadBalancer
```

7.4 Microservices

```
sed "s|REGISTRY|$REGISTRY|g; s|IMAGE_TAG|$TAG|g" k8s/services/deployments.yaml \
| kubectl apply -f -
```

```
for SVC in user-service product-service order-service \
    inventory-service payment-service notification-service; do
    kubectl -n ecommerce rollout status deployment/$SVC --timeout=300s
done
```

7.5 Monitoring

```
kubect1 apply -f k8s/monitoring/monitoring.yaml  
kubect1 -n monitoring get pods
```

7.6 Checkpoint

```
kubectl -n ecommerce get pods
```

Expect **14 pods**, all `Running`: 5 postgres · 1 redis · 1 kafka · 1 kong · 6 services

```
kubectl -n ecommerce get pvc # 7 Bound
```

```
kubectl -n ecommerce get hpa # TARGETS shows numbers, not <unknown>
```

PHASE 8 — Expose via ALB

8.1 The Ingress

```
cat > k8s/ingress.yaml <<EOF
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ecommerce-ingress
  namespace: ecommerce
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80},{"HTTPS":443}]'
    alb.ingress.kubernetes.io/ssl-redirect: '443'
    alb.ingress.kubernetes.io/certificate-arn: $CERT_ARN
    alb.ingress.kubernetes.io/healthcheck-path: /status
    alb.ingress.kubernetes.io/healthcheck-port: '8100'
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: '15'
    alb.ingress.kubernetes.io/healthy-threshold-count: '2'
    alb.ingress.kubernetes.io/unhealthy-threshold-count: '3'
    alb.ingress.kubernetes.io/target-group-attributes: deregistration_delay.timeout_seconds=30
    alb.ingress.kubernetes.io/load-balancer-attributes: idle_timeout.timeout_seconds=60
spec:
  ingressClassName: alb
  rules:
    - host: $DOMAIN
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: kong-proxy
                port:
                  number: 80
EOF
```

```
kubectl apply -f k8s/ingress.yaml
```

```
kubectl -n ecommerce get ingress -w # ADDRESS appears in ~3 min
```

`deregistration_delay=30` is what makes deploys drop zero requests — when a pod terminates the ALB stops sending it *new* traffic but lets in-flight requests finish for 30 seconds.

If ADDRESS stays empty:

```
kubectl -n kube-system logs deploy/aws-load-balancer-controller --tail=50
```

`couldn't auto-discover subnets` means the subnet tags are missing — but eksctl applied them in Phase 2, so verify with the command in 2.3.

8.2 DNS

```
export ALB_DNS=$(kubectl -n ecommerce get ingress ecommerce-ingress \
-o jsonpath='{.status.loadBalancer.ingress[0].hostname}')

export ALB_ZONE=$(aws elbv2 describe-load-balancers --region $AWS_REGION \
--query "LoadBalancers[?DNSName=='$ALB_DNS'].CanonicalHostedZoneId" --output text)

echo "$ALB_DNS / $ALB_ZONE"

aws route53 change-resource-record-sets --hosted-zone-id $ZONE_ID --change-batch "{
  \"Changes\": [{\"Action\": \"UPSERT\", \"ResourceRecordSet\": {
    \"Name\": \"$DOMAIN\", \"Type\": \"A\",
    \"AliasTarget\": {\"HostedZoneId\": \"$ALB_ZONE\", \"DNSName\": \"$ALB_DNS\",
      \"EvaluateTargetHealth\": true}}}]}"

Look the zone ID up, never hardcode it. It's an AWS constant that differs per region, and a wrong value silently breaks the record.

sleep 60 && dig +short $DOMAIN
```

PHASE 9 — Verify

9.1 Health

```
kubectl -n ecommerce get pods,svc,ingress,hpa
kubectl -n ecommerce get pvc

# ALB targets healthy?
TG=$(aws elbv2 describe-target-groups --region $AWS_REGION \
--query "TargetGroups[?contains(TargetGroupName,'ecommerce')].TargetGroupArn" --output text)
aws elbv2 describe-target-health --target-group-arn $TG --region $AWS_REGION \
--query 'TargetHealthDescriptions[].TargetHealth.State' --output text # healthy healthy
```

9.2 End to end

```
export KONG_URL=https://$DOMAIN  
bash .github/smoke-test.sh
```

This exercises **16 endpoints** and pushes a real order through the entire saga to **PAID** — order → Kafka → inventory (Redis lock) → Kafka → payment → notification.

9.3 By hand

```
curl -sX POST https://$DOMAIN/api/users/register \
-H 'Content-Type: application/json' \
-d '{"email":"a@b.com","password":"pass1234","name":"Test"}'

TOKEN=$(curl -sX POST https://$DOMAIN/api/users/login \
-H 'Content-Type: application/json' \
-d '{"email":"a@b.com","password":"pass1234"}' | jq -r .token)

curl -sX POST https://$DOMAIN/api/products \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"name":"Laptop","price":50000,"category":"electronics"}'

curl -sX POST https://$DOMAIN/api/inventory/restock \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"productId":1,"quantity":10}'

curl -sX POST https://$DOMAIN/api/orders \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
-d '{"productId":1,"quantity":2}'

sleep 5
curl -s https://$DOMAIN/api/orders/1 -H "Authorization: Bearer $TOKEN" | jq .status
# "CONFIRMED"
```

Watch the saga while it happens:

```
kubect1 -n ecommerce logs -l app=order-service -f --tail=20
```


9.4 Monitoring

```
kubect1 -n monitoring port-forward svc/grafana 3000:3000 # admin/admin
```

PHASE 10 — Day-2 operations

Deploy a new version

```
export TAG=$(git rev-parse --short HEAD)
# build + push (Phase 4.2), then:
kubect1 -n ecommerce set image deployment/order-service \
  order-service=$REGISTRY/ecommerce/order-service:$TAG
kubect1 -n ecommerce rollout status deployment/order-service
```

Roll back

```
kubect1 -n ecommerce rollout undo deployment/order-service
kubect1 -n ecommerce rollout history deployment/order-service
```

Database migrations do **not** roll back — Flyway runs forward only. During a rolling deploy old and new pods run **simultaneously** against the same database, so every migration must be backward-compatible.

Scale

```
kubect1 -n ecommerce scale deployment/order-service --replicas=5
eksctl scale nodegroup --cluster $CLUSTER --name general --nodes 5
```

Logs

```
kubect1 -n ecommerce logs -l app=order-service --tail=100 -f
kubect1 -n ecommerce logs <pod> --previous # the crashed instance
```

Inspect state

```
kubect1 -n ecommerce exec -it postgres-order-0 -- psql -U postgres -d orderdb \
  -c 'SELECT id,status FROM orders ORDER BY id DESC LIMIT 5;'

kubect1 -n ecommerce exec redis-0 -- redis-cli -a "$REDIS_PW" KEYS '*'

kubect1 -n ecommerce exec kafka-0 -- kafka-console-consumer \
  --bootstrap-server localhost:9092 --topic order.created --from-beginning --max-messages 5
```

PHASE 11 — Cost control

Roughly **\$5-8/day** with this setup:

Item	~Monthly
EKS control plane	\$73
3× t3.large + 2× r5.large	~\$300
2× NAT Gateway (HA)	~\$65
ALB	~\$20
EBS (7 volumes)	~\$15

Cut it while learning:

```
# scale nodes to zero overnight – control plane still bills
eksctl scale nodegroup --cluster $CLUSTER --name general --nodes 0
eksctl scale nodegroup --cluster $CLUSTER --name high-mem --nodes 0
```

Or use `nat.gateway: Single` in `cluster.yaml` (saves ~\$32/month, at the cost of AZ resilience), and `t3.medium` instead of `t3.large`.

Set a billing alarm now: Console → **Billing** → **Budgets** → **Create budget** → Cost budget → \$20/month → alert at 80%.

PHASE 12 — Teardown

Order matters — Kubernetes-created AWS resources must go before the cluster.

```
# 1. Ingress first - this releases the ALB
kubectl delete -f k8s/ingress.yaml
sleep 60

# 2. The app (this also deletes the EBS volumes)
kubectl delete namespace ecommerce monitoring

# 3. The cluster - eksctl removes VPC, subnets, NAT, IGW, node groups, roles
eksctl delete cluster --name $CLUSTER --region $AWS_REGION --wait
```

🕒 ~15 minutes.

Then clean up what eksctl doesn't own:

```
# ECR repositories
for SVC in user-service product-service order-service \
    inventory-service payment-service notification-service; do
    aws ecr delete-repository --repository-name ecommerce/$SVC \
        --region $AWS_REGION --force
done

# certificate
aws acm delete-certificate --certificate-arn $CERT_ARN --region $AWS_REGION

# hosted zone (delete its records first)
aws route53 delete-hosted-zone --id $ZONE_ID
```

Verify nothing is left billing:

```
aws ec2 describe-nat-gateways --region $AWS_REGION \
    --filter Name=state,Values=available --query 'NatGateways[0].NatGatewayId'
aws ec2 describe-addresses --region $AWS_REGION --query 'Addresses[0].AllocationId'
aws elbv2 describe-load-balancers --region $AWS_REGION --query 'LoadBalancers[0].LoadBalancerName'
aws ec2 describe-volumes --region $AWS_REGION \
    --filters Name=status,Values=available --query 'Volumes[0].VolumeId'
```

All four should return empty. **Unattached Elastic IPs and orphaned NAT Gateways are the two that quietly keep charging.**

Troubleshooting

Symptom	Command	Cause
ImagePullBackOff	<code>kubectl -n ecommerce describe pod <p></code>	registry URL (Phase 1.1), or ECR login expired (12h)
PVC Pending	<code>kubectl -n ecommerce describe pvc <p></code>	StorageClass missing (1.2 / 3.1)
PVC Bound, pod won't schedule	<code>kubectl describe pod</code>	volume in wrong AZ — needs <code>WaitForFirstConsumer</code>
Ingress ADDRESS empty	<code>kubectl -n kube-system logs deploy/aws-load-balancer-controller</code>	subnet tags, or controller IAM
ALB up, 503	<code>aws elbv2 describe-target-health --target-group-arn \$TG</code>	Kong not passing <code>/status:8100</code>
CrashLoopBackOff	<code>kubectl logs <pod> --previous</code>	wrong DB credentials, or DB not ready
JWT secret too short	startup logs	<code>JWT_SECRET</code> under 32 chars
Order stuck PENDING	<code>kubectl -n ecommerce logs -l app=inventory-service</code>	Kafka topics missing (7.2)
Redis NOAUTH	any service log	password mismatch between Secret and <code>redis.yaml</code>
HPA <unknown>	<code>kubectl top pods -n ecommerce</code>	metrics-server (3.2) or no CPU requests (1.7)
Emails not arriving	notification logs	SES sandbox, or identity unverified (6.2)
Certificate stuck pending	<code>aws acm describe-certificate ...</code>	validation CNAME missing or DNS not delegated

Full sequence

```

# 0 - setup
export AWS_REGION=ap-south-1 CLUSTER=ecommerce-cluster DOMAIN=yourdomain.com
export ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
export REGISTRY=$ACCOUNT_ID.dkr.ecr.$AWS_REGION.amazonaws.com

# 1 - code fixes (manual, Phase 1)

# 2 - cluster
eksctl create cluster -f cluster.yaml

# 3 - add-ons
eksctl create addon --name aws-ebs-csi-driver --cluster $CLUSTER --force
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
helm install aws-load-balancer-controller eks/aws-load-balancer-controller -n kube-system \
  --set clusterName=$CLUSTER --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller

# 4 - images
export TAG=$(git rev-parse --short HEAD)
aws ecr get-login-password --region $AWS_REGION | docker login --username AWS --password-stdin $REGISTRY
mvn -B -T 1C package -DskipTests
# build + push loop

# 5 - cert + DNS
# 6 - secrets
# 7 - deploy
kubectl apply -f k8s/namespace/ -f k8s/services/secrets.yaml
kubectl apply -f k8s/postgres/ -f k8s/redis/ -f k8s/kafka/ -f k8s/kong/
sed "s|REGISTRY|$REGISTRY|g; s|IMAGE_TAG|$TAG|g" k8s/services/deployments.yaml | kubectl apply -f -

# 8 - expose
kubectl apply -f k8s/ingress.yaml

# 9 - verify
KONG_URL=https://$DOMAIN bash .github/smoke-test.sh

# 12 - teardown
eksctl delete cluster --name $CLUSTER --wait

```